Résolution du CVRP par Colonies de Fourmis améliorée par Q-Learning

Projet d'Optimisation Combinatoire
Apport du Machine Learning pour l'amélioration des métaheuristiques

Grégory MENEUS Arnaud GRIMBERT Odin LANDU

Année universitaire 2025–2026

Résumé

Dans ce rapport, nous présentons notre travail sur la résolution du problème de tournées de véhicules avec contraintes de capacité (CVRP). Nous avons choisi d'implémenter une approche par colonies de fourmis (ACO), puis de l'améliorer en utilisant du Q-Learning pour ajuster automatiquement les paramètres de l'algorithme pendant la recherche. L'idée principale est que, plutôt que de fixer les paramètres de l'ACO une fois pour toutes, on laisse un agent d'apprentissage par renforcement apprendre quelle stratégie adopter selon l'état de la recherche. Nous comparons ensuite les deux approches sur plusieurs instances de tailles différentes, et nous discutons de ce que le machine learning apporte réellement dans ce contexte.

Table des matières

1 Introduction **3**

2 Modélisation du problème **3**
2.1 Description du CVRP . . . 3
2.2 Formulation mathématique . . . 3
2.3 Difficulté et intérêt . . . 4

3 Métaheuristique : Colonies de fourmis (ACO) **4**
3.1 Principe général . . . 4
3.2 Construction d'une solution . . . 4
3.3 Mise à jour des phéromones . . . 5
3.4 Paramètres utilisés . . . 5
3.5 Pseudocode . . . 6

4 Intégration du Q-Learning **6**
4.1 Pourquoi le Q-Learning ? . . . 6
4.2 Formulation état–action–récompense . . . 6
4.2.1 Espace d'états (27 états) . . . 6
4.2.2 Espace d'actions (6 stratégies) . . . 7
4.2.3 Fonction de récompense . . . 8
4.2.4 Mise à jour Q-Learning . . . 8
4.3 Phase de warm-up . . . 8

4.4 Recherche locale 2-opt 8

5 Implémentation **9**
5.1 Architecture du code 9
5.2 Reproductibilité 9

6 Expérimentations **9**
6.1 Protocole expérimental 9
6.2 Résultats 10
6.3 Analyse des résultats 10
6.3.1 Qualité des solutions 10
6.3.2 Stabilité 10
6.3.3 Temps de calcul 11
6.4 Visualisation de la convergence 11

7 Discussion **11**
7.1 Apports du Q-Learning 11
7.2 Limites 12
7.3 Pistes d'amélioration 12

8 Conclusion **13**

1 Introduction

Les problèmes d'optimisation combinatoire sont partout dans le monde réel : logistique, planification, allocation de ressources, etc. Parmi eux, le problème de tournées de véhicules (VRP) est un classique, et sa variante avec capacités (CVRP) est celle que l'on rencontre le plus souvent en pratique.

Le CVRP consiste à trouver les meilleurs itinéraires pour une flotte de véhicules qui doivent livrer des clients à partir d'un dépôt central. Chaque véhicule possède une capacité maximale à ne pas dépasser. Il s'agit d'un problème NP-difficile, ce qui signifie qu'il n'existe pas d'algorithme capable de trouver la solution optimale en temps raisonnable dès que le nombre de clients augmente.

Pour résoudre ce type de problème, on utilise souvent des métaheuristiques. Ces algorithmes ne garantissent pas de trouver l'optimum, mais ils fournissent de bonnes solutions en un temps acceptable. Cependant, leurs performances dépendent fortement de leurs paramètres, et les régler correctement est souvent un travail fastidieux qui se fait par essais-erreurs.

C'est là que le machine learning entre en jeu. L'idée que nous explorons dans ce projet est d'utiliser un algorithme d'apprentissage par renforcement (le Q-Learning) pour que l'algorithme apprenne lui-même à ajuster ses paramètres au fil de la résolution. Au lieu de chercher manuellement les "bons" paramètres, nous laissons l'algorithme s'adapter dynamiquement.

Dans ce rapport, nous commençons par présenter le CVRP et sa modélisation. Ensuite, nous décrivons notre implémentation de l'ACO classique, puis l'intégration du Q-Learning. Enfin, nous présentons nos résultats expérimentaux et discutons de ce qui fonctionne et de ce qui fonctionne moins bien.

2 Modélisation du problème

2.1 Description du CVRP

Le CVRP (Capacitated Vehicle Routing Problem) se définit comme suit :

— Un **dépôt central** d'où partent et reviennent tous les véhicules.
— n **clients**, chacun situé à une position (x_i, y_i) et ayant une demande d_i à satisfaire.
— Une flotte de véhicules identiques, chacun avec une **capacité maximale** Q.
— Chaque client doit être visité **exactement une fois** par un seul véhicule.
— La charge totale d'un véhicule sur sa tournée ne doit pas dépasser Q.

2.2 Formulation mathématique

Nous modélisons le problème sur un graphe complet $G = (V, E)$ où $V = \{0, 1, \ldots, n\}$ (0 étant le dépôt) et E l'ensemble des arêtes. La distance entre deux nœuds i et j est la distance euclidienne :

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \tag{1}$$

Une solution est un ensemble de routes $R = \{r_1, r_2, \ldots, r_K\}$ où chaque route r_k est une séquence de clients qui commence et finit au dépôt. La fonction objectif à minimiser est la distance totale :

$$\min \sum_{k=1}^{K} \left(d_{0,r_k(1)} + \sum_{i=1}^{|r_k|-1} d_{r_k(i),r_k(i+1)} + d_{r_k(|r_k|),0} \right) \tag{2}$$

sous les contraintes :

$$\sum_{i \in r_k} d_i \leq Q \quad \forall k \in \{1, \ldots, K\} \tag{3}$$

$$\bigcup_{k=1}^{K} r_k = \{1, \ldots, n\} \tag{4}$$

$$r_k \cap r_l = \emptyset \quad \forall k \neq l \tag{5}$$

2.3 Difficulté et intérêt

Le CVRP est NP-difficile. Concrètement, pour n clients, le nombre de solutions possibles explose de façon factorielle. Même avec 50 clients, il est impossible d'énumérer toutes les solutions. La métaheuristiques va nous permettre d'explorer intelligemment ce très grand espace.

Le CVRP présente aussi un intérêt réel. Il correspond au problème que rencontrent les entreprises de livraison, comment organiser les tournées pour minimiser les coûts tout en respectant la capacité des camions ?

3 Métaheuristique : Colonies de fourmis (ACO)

3.1 Principe général

L'optimisation par colonies de fourmis (ACO), introduite par Dorigo et Di Caro [2], s'inspire du comportement des fourmis dans la nature. Quand des fourmis cherchent de la nourriture, elles déposent des phéromones sur leur chemin. Les chemins les plus courts accumulent plus de phéromones (parce que les fourmis y passent plus souvent), ce qui attire davantage de fourmis : c'est un mécanisme d'auto-renforcement.

Dans notre cas, les "fourmis" sont des agents qui construisent des solutions au CVRP. Elles choisissent le prochain client à visiter en se basant sur deux critères :

— Les **phéromones** τ_{ij} : plus il y a de phéromone sur un arc, plus il est attractif (c'est la mémoire collective).

— L'**heuristique** $\eta_{ij} = 1/d_{ij}$: les clients proches sont naturellement préférés.

3.2 Construction d'une solution

Chaque fourmi construit une solution complète. Elle part du dépôt et, tant qu'il reste des clients non visités, elle choisit le prochain client selon la probabilité :

$$p_{ij} = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in \mathcal{N}_i} [\tau_{il}]^\alpha \cdot [\eta_{il}]^\beta} \tag{6}$$

où $\mathcal{N}_i$ est l'ensemble des clients encore accessibles (non visités et dont la demande ne fait pas dépasser la capacité), α contrôle l'importance des phéromones et β l'importance de l'heuristique.

Quand une fourmi ne peut plus ajouter de client à sa route courante (à cause de la capacité), elle retourne au dépôt et commence une nouvelle route.

3.3 Mise à jour des phéromones

Après que toutes les fourmis aient construit leur solution, on met à jour les phéromones en deux étapes :

Évaporation : les phéromones diminuent sur tous les arcs pour éviter de rester bloqué sur de mauvaises solutions :

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} \tag{7}$$

Dépôt (stratégie élitiste) : seules les meilleures fourmis (le tiers supérieur) déposent des phéromones, et la meilleure solution globale reçoit un bonus supplémentaire :

$$\tau_{ij} \leftarrow \tau_{ij} + \sum_{k \in \text{élite}} w_k \cdot \frac{Q_{\text{phéro}}}{C_k} \tag{8}$$

où C_k est le coût de la solution de la fourmi k et w_k un poids décroissant selon le rang.

3.4 Paramètres utilisés

Le tableau 1 résume les paramètres de notre ACO.

TABLE 1 – Paramètres de l'ACO classique

Paramètre	**Valeur**	**Rôle**
α	1.0	Poids des phéromones
β	3.0	Poids de l'heuristique
ρ	0.1	Taux d'évaporation
$Q_{\text{phéro}}$	100	Facteur de dépôt
$\tau_{\min}$	0.1	Phéromone minimale
$\tau_{\max}$	10.0	Phéromone maximale
Nombre de fourmis	20	Par itération
Itérations max	150	Critère d'arrêt

Nous avons choisi $\beta > \alpha$ pour que l'heuristique de distance ait plus d'influence que les phéromones au début, lorsque celles-ci ne contiennent pas encore beaucoup d'information. Le taux d'évaporation $\rho = 0.1$ signifie que 10% des phéromones disparaissent à chaque itération : une valeur trop faible (ex. 0.01) figerait la recherche sur les premiers chemins trouvés, tandis qu'une valeur trop forte (ex. 0.5) effacerait l'information trop vite pour que les bons chemins se renforcent. La valeur 0.1 offre un équilibre que nous avons vérifié expérimentalement lors de nos tests préliminaires.

3.5 Pseudocode

Algorithm 1 ACO classique pour le CVRP

Entrées : Instance CVRP, paramètres α, β, ρ, Q
Sorties : Meilleure solution trouvée

```
Initialiser la matrice de phéromones τ_ij = 1.0
pour t = 1 à T_max faire
    pour chaque fourmi k faire
        Construire une solution S_k (choix probabiliste + contrainte de capacité)
    fin pour
    Identifier les meilleures solutions (élite)
    Évaporer : τ_ij ← (1 − ρ) · τ_ij
    Déposer : phéromones par les fourmis élites + bonus meilleur global
    Mettre à jour la meilleure solution globale
fin pour
retourner meilleure solution
```

4 Intégration du Q-Learning

4.1 Pourquoi le Q-Learning ?

L'idée de combiner du Q-Learning avec une métaheuristique est quelque chose que nous avions dès le départ du projet. En cherchant dans la littérature, nous sommes tombés sur le survey de Yang et al. [1] qui recense une vingtaine d'années de travaux sur le sujet. C'est en lisant ce papier et en faisant différentes recherches sur le web que nous avons structuré notre approche. Ce survey recense l'adaptation de paramètres par Q-Learning (voir section 4.1.1 de [1]), la sélection d'opérateurs guidée par la Q-table (section 4.1.5), et l'utilisation de politiques ε-greedy pour gérer l'équilibre exploration/exploitation (section 4.1.4). Notre approche reprend ces principes et les adapte au contexte de l'ACO pour le CVRP.

Nous avons choisi le Q-Learning [3] pour plusieurs raisons. D'abord, c'est un algorithme relativement simple à comprendre et à implémenter. Ensuite, notre problème d'ajustement de paramètres se formule naturellement comme un problème de décision séquentielle : à chaque itération, on observe l'état de la recherche et on décide quelle stratégie adopter.

Par rapport à une simple grille de recherche de paramètres (grid search), le Q-Learning a l'avantage d'être adaptatif : il peut changer de stratégie au cours de la résolution, par exemple explorer au début puis intensifier vers la fin.

4.2 Formulation état–action–récompense

4.2.1 Espace d'états (27 états)

L'état de la recherche est caractérisé par trois dimensions, que nous discrétisons chacune en 3 niveaux :

- **Taux d'amélioration** : mesure si la recherche progresse ou stagne.
  - Stagnation ($\leq 0.1\%$), faible amélioration ($\leq 1\%$), forte amélioration ($> 1\%$)
- **Diversité** : coefficient de variation des coûts des solutions de l'itération.
  - Convergé ($\leq 5\%$), moyen ($\leq 15\%$), diversifié ($> 15\%$)

— **Phase de recherche** : avancement en pourcentage des itérations.
  — Début ($\leq 33\%$), milieu ($\leq 66\%$), fin ($> 66\%$)

Ce qui donne $3 \times 3 \times 3 = 27$ états possibles. Nous avons retenu ces caractéristiques car elles capturent les aspects essentiels de la recherche : l'algorithme progresse-t-il, explore-t-il suffisamment, et à quelle phase de la résolution se trouve-t-on.

4.2.2 Espace d'actions (6 stratégies)

Plutôt que de faire varier les paramètres un par un (augmenter α de 0.1, diminuer ρ de 0.05, etc.), On défini 6 stratégies qui correspondent à des profils de paramètres cohérents. Ce choix se justifie par le fait que les paramètres α, β et ρ dépendent les uns des autres : augmenter α sans ajuster ρ en conséquence produit un comportement incohérent. De plus, des ajustements incrémentaux nécessiteraient un nombre d'actions bien plus important, ce qui ralentirait la convergence du Q-Learning dans notre espace limité de 150 itérations.

Table 2 – Stratégies disponibles pour le Q-Learning

Stratégie	**Objectif**	α	β	ρ
`exploitation`	Suivre les phéromones	2.0	2.5	0.05
`exploration`	Explorer de nouvelles routes	0.5	4.0	0.20
`balanced`	Équilibre	1.0	3.0	0.10
`intensify`	Renforcer les meilleurs chemins	2.5	2.0	0.02
`diversify`	Relancer la recherche	0.3	5.0	0.30
`local_search`	Améliorer par 2-opt	1.0	3.0	0.10

Les valeurs numériques de chaque profil découlent directement de la logique de la formule de probabilité $p_{ij} \propto \tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}$:

— **Exploitation** ($\alpha = 2.0$, $\beta = 2.5$, $\rho = 0.05$) : un α élevé amplifie l'écart entre les arcs très phéromonés et les autres, ce qui force les fourmis à suivre les chemins déjà établis. Le ρ faible conserve ces phéromones plus longtemps.

— **Exploration** ($\alpha = 0.5$, $\beta = 4.0$, $\rho = 0.20$) : un α faible réduit l'influence des phéromones, donc les fourmis ne sont plus attirées par les chemins connus. Le β élevé les guide quand même vers les clients proches (pour éviter des solutions absurdes), et le ρ élevé efface rapidement les anciennes traces.

— **Balanced** ($\alpha = 1.0$, $\beta = 3.0$, $\rho = 0.10$) : ce sont les paramètres par défaut de notre ACO, un équilibre entre phéromones et heuristique.

— **Intensify** ($\alpha = 2.5$, $\beta = 2.0$, $\rho = 0.02$) : version plus agressive que `exploitation`. Le ρ très faible (2% d'évaporation) conserve les phéromones au maximum pour renforcer les meilleurs chemins trouvés.

— **Diversify** ($\alpha = 0.3$, $\beta = 5.0$, $\rho = 0.30$) : version plus agressive que `exploration`. Le ρ à 0.30 efface 30% des phéromones par itération, ce qui "remet à zéro" la mémoire collective et force la découverte de nouvelles zones.

— **Local_search** ($\alpha = 1.0$, $\beta = 3.0$, $\rho = 0.10$) : paramètres équilibrés, mais déclenche en plus le 2-opt (voir section suivante).

Les plages de valeurs ($\alpha \in [0.3, 2.5]$, $\beta \in [2.0, 5.0]$, $\rho \in [0.02, 0.30]$) ont été déterminées par des tests préliminaires : en dehors de ces bornes, l'algorithme produit soit des solutions de mauvaise qualité ($\alpha > 3$ fige la recherche), soit une convergence trop lente ($\rho > 0.5$ empêche tout apprentissage collectif).

L'idée derrière ces stratégies est intuitive. Lorsque la recherche stagne et que les solutions convergent trop, il vaut mieux diversifier ou explorer. Lorsque l'on se trouve dans une bonne zone de l'espace de recherche, on souhaite intensifier. Et lorsque les solutions sont de bonne qualité, le 2-opt peut les améliorer localement.

La stratégie `local_search` est particulière : en plus de fixer des paramètres équilibrés, elle déclenche un opérateur de recherche locale 2-opt sur chaque solution construite par les fourmis.

4.2.3 Fonction de récompense

La récompense est calculée à chaque itération pour évaluer si la stratégie choisie a été bénéfique :

$$r_t = \begin{cases} 1.0 + 10 \cdot \frac{C_{best}^{t-1} - C_{best}^{t}}{C_{best}^{t-1}} & \text{si amélioration du meilleur coût} > 0.1\% \\ 0.5 \cdot \frac{\bar{C}^{t-1} - \bar{C}^{t}}{\bar{C}^{t-1}} & \text{si amélioration du coût moyen seulement} \\ -0.05 & \text{sinon (stagnation)} \end{cases} \tag{9}$$

Nous avons choisi une pénalité faible pour la stagnation (-0.05) plutôt qu'une forte pénalité, car stagner n'est pas nécessairement problématique : l'algorithme peut déjà être proche de l'optimum. La récompense positive est proportionnelle à l'amélioration afin d'encourager les améliorations significatives.

4.2.4 Mise à jour Q-Learning

La mise à jour suit la règle standard du Q-Learning [3] :

$$Q(s,a) \leftarrow Q(s,a) + \alpha_{lr} \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{10}$$

avec $\alpha_{lr} = 0.2$ (taux d'apprentissage) et $\gamma = 0.95$ (facteur d'actualisation).

La politique de sélection est ε-greedy : avec probabilité ε on choisit une action aléatoire (exploration), sinon on prend la meilleure selon la table Q. ε commence à 0.15 et décroît progressivement jusqu'à 0.02.

4.3 Phase de warm-up

Un détail important de notre implémentation : nous ne laissons pas le Q-Learning intervenir dès le début. Pendant les 10% premières itérations (soit 15 itérations sur 150), l'ACO tourne avec ses paramètres par défaut. Cela permet d'établir une base de phéromones raisonnable avant de commencer à modifier les paramètres. Sans cette phase de warm-up, le Q-Learning risque de prendre de mauvaises décisions faute d'information suffisante sur l'état de la recherche.

4.4 Recherche locale 2-opt

Le 2-opt est un opérateur d'amélioration locale assez courant. L'idée dans notre cas est simple : on sélectionne deux arcs dans une route et on inverse le segment entre eux pour vérifier si cela réduit la distance totale. Ce processus est répété tant que des améliorations sont trouvées.

Nous l'appliquons uniquement lorsque le Q-Learning choisit la stratégie `local_search`. Ce choix est délibéré : le 2-opt est coûteux en temps de calcul, et nous ne souhaitons donc pas l'appliquer à chaque itération. C'est au Q-Learning de décider quand son application est pertinente (voir section 4.1.5 de [1] pour d'autres exemples de sélection d'opérateurs par Q-Learning).

5 Implémentation

5.1 Architecture du code

Le projet est implémenté en Python. Nous avons organisé le code en modules séparés pour maintenir une bonne lisibilité :

— `src/cvrp.py` : modélisation du problème (instances, solutions, distances)
— `src/aco.py` : implémentation de l'ACO classique
— `src/qlearning.py` : agent Q-Learning
— `src/hybrid_aco_ql.py` : solveur hybride qui combine les deux
— `src/local_search.py` : opérateur 2-opt
— `src/experiments.py` : lancement des expériences
— `app.py` : interface web Flask pour la démonstration

Nous avons utilisé NumPy pour les calculs sur la table Q et Flask pour l'interface de démonstration. L'interface web permet de configurer les paramètres, de lancer une résolution, et de visualiser les routes sur une carte avec les courbes de convergence.

5.2 Reproductibilité

Toutes les expériences utilisent des graines aléatoires (seeds) fixes, ce qui permet de reproduire exactement les mêmes résultats. Le code est disponible sur GitHub et peut être lancé soit directement avec Python, soit via Docker.

6 Expérimentations

6.1 Protocole expérimental

Nous avons testé notre approche sur 7 instances générées aléatoirement avec des tailles croissantes (10 à 50 clients). Pour chaque instance, nous effectuons 5 exécutions avec des graines aléatoires (seeds) différentes afin d'obtenir des résultats statistiques. Nous comparons l'ACO seul et l'ACO hybride (avec Q-Learning) sur les mêmes instances avec les mêmes seeds.

Les expériences sont exécutées avec 20 fourmis et 150 itérations, un compromis entre qualité des résultats et temps de calcul raisonnable.

Note : les instances utilisées sont générées aléatoirement et ne proviennent pas de benchmarks standard (tels que VRPLIB). Par ailleurs, avec seulement 5 exécutions par instance, nous ne réalisons pas de test statistique formel (test de Student, etc.). Les améliorations rapportées sont donc indicatives et mériteraient une validation plus rigoureuse.

6.2 Résultats

Le tableau 3 présente nos résultats. Désolé si il n'est pas bien centré il y a beaucoup de colones.

TABLE 3 – Comparaison ACO classique vs ACO + Q-Learning

Instance	n	**ACO best**	**ACO moy ± std**	**Hybride best**	**Hybride moy ± std**	**Amél.**
small_10	10	562.6	562.6 ± 0.0	562.6	562.6 ± 0.0	0.00%
small_15	15	478.5	481.5 ± 1.5	478.5	480.4 ± 1.6	+0.23%
medium_20	20	498.2	501.3 ± 5.5	498.2	504.1 ± 7.1	−0.55%
medium_25	25	771.1	781.0 ± 10.2	770.2	772.8 ± 3.6	+1.05%
medium_30	30	732.1	741.1 ± 9.0	722.7	733.8 ± 5.9	+0.99%
large_40	40	827.6	858.8 ± 17.9	817.0	828.9 ± 12.2	**+3.49%**
large_50	50	921.1	933.0 ± 8.7	889.5	920.2 ± 25.7	+1.37%

6.3 Analyse des résultats

6.3.1 Qualité des solutions

On observe plusieurs phénomènes intéressants dans ces résultats (la colonne "Amél." correspond à l'amélioration relative du coût moyen) :

- Sur les petites instances (10–15 clients), les deux approches donnent des résultats quasiment identiques. Cela est cohérent : l'espace de recherche est restreint, et l'ACO classique trouve déjà de très bonnes solutions. Le Q-Learning n'a pas de marge d'amélioration significative.
- Sur les instances moyennes (20–30 clients), les résultats sont mitigés. Sur 20 clients, l'hybride est légèrement moins performant en moyenne (−0.55%), mais sur 25 et 30 clients, il fait mieux (+1.05% et +0.99%). Le résultat négatif sur 20 clients s'explique probablement par le fait que l'exploration du Q-Learning introduit de la variance sans apporter suffisamment de bénéfice sur une instance de cette taille.
- Sur les grandes instances (40–50 clients), l'hybride montre clairement son avantage. Le meilleur résultat est obtenu sur 40 clients avec +3.49% d'amélioration. Cela est cohérent avec l'intuition : plus l'espace de recherche est vaste, plus l'ajustement dynamique des paramètres peut aider à explorer différentes zones.

6.3.2 Stabilité

Un point notable : sur les instances moyennes (25 et 30 clients), l'écart-type de l'hybride est plus faible que celui de l'ACO seul (3.6 vs 10.2 pour 25 clients). Cela suggère que le Q-Learning aide à stabiliser la recherche en choisissant des stratégies adaptées. Cependant, sur les grandes instances, l'écart-type augmente (25.7 vs 8.7 pour 50 clients), ce qui indique que l'exploration du Q-Learning peut également introduire de la variabilité.

6.3.3 Temps de calcul

TABLE 4 – Comparaison des temps de calcul moyens (en secondes)

Instance	ACO	Hybride	Surcoût
small_10	0.14 s	0.17 s	+27%
small_15	0.27 s	0.34 s	+27%
medium_20	0.46 s	0.59 s	+27%
medium_25	0.64 s	0.80 s	+24%
medium_30	0.91 s	1.18 s	+29%
large_40	1.51 s	1.88 s	+24%
large_50	2.27 s	2.82 s	+24%

Le surcoût du Q-Learning est d'environ 25–29%. Ce n'est pas négligeable, mais cela reste raisonnable. Ce surcoût provient principalement du 2-opt, qui est déclenché par la stratégie `local_search`. Le Q-Learning lui-même (consultation de la table Q et mise à jour) est très rapide et ajoute un surcoût négligeable.

6.4 Visualisation de la convergence

La figure 1 présente la convergence réelle des deux approches sur l'instance `large_40` (40 clients, seed 100). On constate que l'ACO classique converge rapidement vers un plateau autour de l'itération 30 et stagne ensuite. L'hybride, grâce aux changements de stratégie pilotés par le Q-Learning, parvient à rompre cette stagnation à plusieurs reprises (notamment autour des itérations 55 et 75) et atteint un coût final sensiblement inférieur.

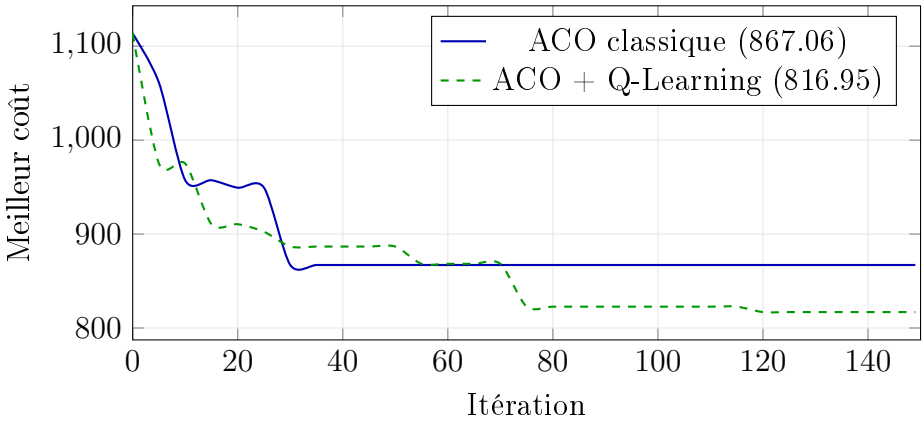

FIGURE 1 – Convergence réelle sur l'instance `large_40` (seed 100)

7 Discussion

7.1 Apports du Q-Learning

D'après nos expériences, le Q-Learning apporte plusieurs contributions :

1. **Adaptation automatique des paramètres** : c'est le point principal. Plutôt que de chercher manuellement les bons paramètres, le Q-Learning apprend à adapter

la stratégie. Cela fonctionne surtout lorsque l'espace de recherche est suffisamment grand pour que le choix de stratégie fasse une réelle différence.

2. **Meilleure exploration sur les grandes instances** : sur 40 et 50 clients, l'hybride trouve de meilleures solutions. Le Q-Learning parvient à alterner entre exploration et exploitation de façon pertinente.
3. **Recherche locale ciblée** : le fait que le Q-Learning puisse déclencher le 2-opt au bon moment est un atout. Appliquer le 2-opt systématiquement serait trop coûteux, mais le faire lorsque c'est pertinent apporte un gain appréciable.

7.2 Limites

Notre approche a aussi des limites :

1. **Pas d'amélioration sur les petites instances** : lorsque le problème est suffisamment simple pour que l'ACO classique trouve de bonnes solutions seul, le Q-Learning n'apporte rien et peut même légèrement dégrader les performances (cas des 20 clients).
2. **Pas de transfert d'apprentissage** : notre agent Q-Learning apprend pendant la résolution d'une instance et repart de zéro pour l'instance suivante. Il n'y a pas de mémoire d'une instance à l'autre, ce qui signifie qu'il doit réapprendre à chaque fois.
3. **Discrétisation grossière** : notre espace d'états ne comporte que 27 états. C'est assez grossier et on perd nécessairement de l'information. De plus, la métrique de diversité (coefficient de variation des coûts) ne capture que la diversité en termes de coût et non la diversité structurelle des routes. Cependant, avec davantage d'états, il faudrait plus d'itérations pour que le Q-Learning converge, ce qui constitue un compromis.
4. **Surcoût en temps** : le surcoût de 25% n'est pas critique, mais il existe. Sur des instances très grandes ou sous des contraintes de temps serrées, cela pourrait poser problème.
5. **Variance accrue sur les grandes instances** : l'écart-type de l'hybride est plus élevé sur 50 clients, ce qui signifie que les résultats sont moins prévisibles.

7.3 Pistes d'amélioration

Plusieurs pistes pourraient permettre d'améliorer notre approche :

— **Transfert d'apprentissage** : pré-entraîner la table Q sur un ensemble d'instances pour que l'agent dispose d'une bonne politique de départ lorsqu'il aborde une nouvelle instance. Cela pourrait réduire la phase d'exploration initiale.

— **Plus d'actions** : ajouter d'autres opérateurs (3-opt, relocate, swap entre routes) comme actions du Q-Learning pour enrichir le répertoire de stratégies.

— **Deep Q-Learning** : remplacer la table Q par un réseau de neurones pour gérer un espace d'états plus fin (features continues au lieu de discrétiser). Cela permettrait d'être plus précis dans la caractérisation de l'état de la recherche.

— **Multi-agent** : utiliser plusieurs agents Q-Learning, chacun gérant un sous-ensemble de paramètres, pour une granularité plus fine.

8 Conclusion

Dans ce projet, nous avons implémenté un solveur hybride pour le CVRP qui combine une métaheuristique classique (ACO) avec un algorithme d'apprentissage par renforcement (Q-Learning). L'objectif était de déterminer si le machine learning pouvait améliorer les performances de l'ACO en ajustant automatiquement ses paramètres.

Nos résultats montrent que l'approche hybride apporte un gain sur les instances de taille suffisante (25+ clients), avec des améliorations allant jusqu'à 3.5% sur le coût moyen. Le Q-Learning parvient à identifier des stratégies pertinentes selon l'état de la recherche, notamment en alternant entre exploration et exploitation. L'intégration de la recherche locale 2-opt comme action pilotée par le Q-Learning constitue également un apport intéressant.

En revanche, sur les petites instances, le Q-Learning n'apporte pas de gain significatif. Il s'agit d'un résultat attendu : lorsque le problème est simple, il est préférable de conserver une approche simple. L'hybride montre son intérêt lorsque l'espace de recherche est suffisamment complexe pour qu'une stratégie adaptative fasse la différence.

Ce projet nous a permis d'observer concrètement comment le machine learning peut interagir avec une métaheuristique. Nous pensons qu'avec des techniques plus avancées (deep RL, transfert d'apprentissage) et une validation sur des benchmarks standards, il serait possible d'aller plus loin.

Références

[1] Y. Yang, Y. Gao, Z. Ding, J. Wu, S. Zhang, F. Han, X. Qiu, S. Gao et Y.-G. Wang, "Advancements in Q-learning meta-heuristic optimization algorithms : A survey," *WIREs Data Mining and Knowledge Discovery*, vol. 14, no. 6, e1548, 2024. DOI : https://doi.org/10.1002/widm.1548

[2] M. Dorigo et G. Di Caro, "Ant colony optimization : A new meta-heuristic," in *Proceedings of the 1999 Congress on Evolutionary Computation (CEC'99)*, vol. 2, pp. 1470–1477, IEEE, 1999.

[3] C. J. C. H. Watkins et P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.